

Approach to build a class of Software Systems through a Combination of Design Patterns

Dimitrichka Zheleva Nikolaeva and Violeta Todorova Bozhikova

Department of Software and Internet Technologies, Faculty of Computer Sciences and Automation,
Technical University of Varna, 1 Studentska str., 9010 Varna, Bulgaria
dima.nikolaeva@abv.bg, vbozhikova2000@yahoo.com

Abstract – Design Patterns (DPs) are recognized as a tool that could be useful in solving repetitive problems that have arisen in real programming practice. The paper summarizes the combined use of DPs and the benefits of using DPs in software building, makes as a summary of Compound Design Patterns (CDP) classifications and presents an Approach to build a class of Software Systems through a Combination of DPs. The paper also presents real results from the use of the approach, namely the realization of an Information System (IS) for the purposes of Maritime Transport.

Keywords – Design Patterns, Combination of Design Patterns, Software Reuse, Software Engineering

I. INTRODUCTION

The strictness of business in creating new applications used for various business purposes places developers ahead of real problems with the speed and automation of the application development process. One possible approach to resolving this problem is using DPs. These are solutions to common problems in programming practice. Their use is done both on their own and in combination with other CDP [11]. This helps to reuse the program code, which increases the productivity of software developers [1-18].

The combined use of DPs for software development has the following advantages:

- reuse of code and automation applicable to a particular software development or its expansion [13];
- flexible, extensible and modular application design [1,2];
- quickly and easily built applications, due to the possibility of reuse;
- modularity and better reuse of the code. The code becomes easier to maintain, the cost is lower, and the time for marketing is shorter. [14]

II. COMBINED USE OF DESIGN PATTERNS.

CDP have the effects of using multiple patterns together. The questions which arise regarding to their implementation are: Which are the possible combinations of DPs? How do DPs connect to each other? How are DPs classified according to their relationships? How are they applied in practice to build software? In Table 1 presents a sample of literature source analysis using Gang of Four (GOF) DPs combinations [12].

TABLE 1

DP	№ article												ABCDP
	1	2	3	6	9	10	15	16	16	16	16	16	
1													X
2					X								
3		X			X	X							
4													
5					X								X
6						X							
7													
8	X		X	X									
9	X			X						X			
10													X
11					X								
12						X					X		
13													
14													X
15									X				
16					X								
17													
18					X			X					
19								X					
20					X				X	X	X		
21	X	X	X	X									
22													
23			X										
24						X							

Remark: Abstract factory¹, Builder², Factory method³, Prototype⁴, Singleton⁵, Adapter⁶, Bridge⁷, Composite⁸, Decorator⁹, Facade¹⁰, Flyweight¹¹, Proxy¹², Chain of responsibility¹³, Command¹⁴, Interpreter¹⁵, Iterator¹⁶, Mediator¹⁷, Memento¹⁸, Observer¹⁹, State²⁰, Strategy²¹, Template method²², Visitor²³, Others²⁴, ABCDP – Approach based on a Combination of Design Patterns

The last column in Table 1 shows the combination of DPs used in the author's proposed Approach for developing a class of systems based on a Combination of DPs discussed in Section 3 of this report.

The analysis of literary sources [4, 5, 7, 9] allows the following CDP classification criteria:

- According to the relations between the DPs [4] (Table2):

- Pattern X uses Pattern Y in its decision;
- Pattern X is similar to Pattern Y;
- Pattern X can be combined with Pattern Y.

- According to the similarity between DPs [5, 9]:

- Factory Method, Abstract Factory, Builder;
- Singleton, static class;
- Abstract Factory, Singleton, Factory Method;
- Prototype, Abstract Factory, Pool Object;
- Multiton/ Registry of Singleton, Pool Singleton, Singleton and Pool Object;
- Adapter, Façade.

- Depending on the overlap between the DPs [7]:

- Conservative Combination of Patterns;
- Combination of Overlap Patterns.

The conservative combination maintains the separation of the DPs structure involved so that they can always be identified in the programmer code. On the other hand,

when DP overlap, the elements of these Patterns merge into one structure. It is not possible to trace which elements overlap due to partial or complete merging of DPs elements.

TABLE 2 DESIGN PATTERN RELATIONSHIPS

A		B		C	
X	Y	X	Y	X	Y
2	1	1	6	8	13
3	2	1	7	7	3
1	8	6	7	3	16
6	11	7	17	3	18
12	11	2	17	16	18
9	3	16	17	3	10
9	16	23	17	21	23
9	18	4	5	-	-
18	19	4	10	-	-
20	3	5	10	-	-
20	19	22	23	-	-
21	3	14	15	-	-
21	10	-	-	-	-
22	5	-	-	-	-

Remark:
A. Pattern X uses Pattern Y in its decision;
B. Pattern X is similar to Pattern Y;
C. Pattern X can be combined with Pattern Y.
Abstract factory¹, Flyweight², Composite³, Proxy⁴, Adapter⁵, Prototype⁶, Builder⁷, Factory method⁸, Interpreter⁹, Decorator¹⁰, Solitaire¹¹, Observer¹², Template method¹³, Glue¹⁴, Mediator¹⁵, Visitor¹⁶, Strategy¹⁷, Iterator¹⁸, Memento¹⁹, Command²⁰, Chain of responsibility²¹, Bridge²², State²³

III. APPROACH TO DEVELOP A CLASS OF SOFTWARE SYSTEMS BASED ON A COMBINATION OF DESIGN PATTERNS.

These are IS with the following common functionality:

- Access to Relational Database (RDB);
- Encryption and decryption of data;
- E-mail delivery;
- Working with basic and auxiliary menus;
- Entering and modifying data in RDB;
- Generating reports.

Suggested approach (Fig.1) is a combination of DPs from GOF: Singleton, Abstract Factory, Façade and Command. There are 4 Patterns (Table 1, Column ABCDP) to solve 6 types of tasks:

- A. To work with User Registry, Encryption and Decryption of Data - Abstract Factory Design Pattern;
- B. To access the RDB - Singleton Design Pattern;
- C. To use the menus - Command Design Pattern;
- D. For entering and modifying data in RDB - Abstract Factory Design Pattern;
- E. Data input and calculation - Façade Design Pattern;
- F. To send mails - Façade Design Pattern.

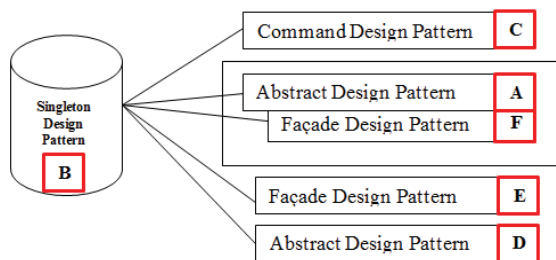


Fig.1 Generic description of the approach: software, based on a Combination of DP is produced

Following this approach (Fig.1), we could develop many other Information Systems.

IV. IMPLEMENTATION OF AN INFORMATION SYSTEM FOR MARITIME TRANSPORT PURPOSES, USING THE PROPOSED APPROACH.

Using already developed approach to creating software based on a combination of DPs, an IS for Maritime Transport was created.

The techniques and tools used to develop an IS for Maritime Transport purposes include: The relational database implemented through SQL Server, a Visual Studio and the C# programming language to create the application itself.

IS has two types of users - administrator and operator. Operators have several levels of access to view content (full rights), modify and edit data (limited rights). The administrator has full access rights to each module from the system, as well as the ability to delete records.

A. FUNCTIONALITY OF IS.

An approach based on a Combination of DPs is applied in the developed IS. For this purpose, a combination of 10 DPs based on the GOF's DPs (Singleton, Abstract Factory, Façade and Command) was developed:

```

DPAbstractFactoryDestination.cs
DPAbstractFactoryLogbook.cs
DPAbstractFactoryMovementBegin.cs
DPAbstractFactoryMovementEnd.cs
DPAbstractFactorySHA256.cs
DPAbstractFactoryShip.cs
DPCommandMenu.cs
DPConsumptionCalcFacade.cs
DPFacadeMail.cs
DPSingleton.cs

```

Fig.2 Classes with DP

The main sections of the software system are presented in Fig.3.

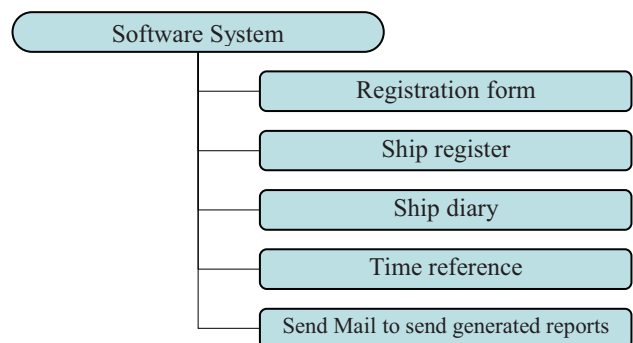


Fig.3 Basic structural elements of the software system

- Registration form gives ability to: add new user and Change of forgotten password.
- Ship register allows to input basic data about: the ship, the destination, for movements begin and end.
- A Ship diary calculates the consumption of fuels, oils and potable water.

- Time reference: five types of reports are automatically generated for the purpose of the application. First, select a report type and set a date range, then generate one of the reports: NOON, End of Sea Passage, Commence of Sea Passage, CAST OF or Finish with Engine. The generated report is in .xmls format. The Report NOON can be seen from Table 3.

TABLE 3 NOON REPORT

m/v Tarifa				
C/E NOON REPORT 5/15/2018 12:00				
M/E rpm	86.9 rpm	87.0 St. time	24.0 hrs	
Brg. Dist	398.00 nm	Average Eng. Dist.	435.8 nm	
Slip	8.7% P	8.6% Av. Power	%	
M/E speed	18.16 kts	M/E Power	10550.9 BHP	
M/E Power	7868 kw	49.7%		
HSFO Consumption		LSDO Consumption		
M/E	40.4 m/t	M/E	0.00 m/t	
D/G	4.60 m/t	D/G	0.00 m/t	
Boiler	0.00 m/t	Boiler	0.00 m/t	
Total	45.0 m/t	Total	0.00 m/t	
LO Consumption		LSGO Consumption		
M/E	20 Ltrs	M/E	0.00 m/t	
M/E Cyl. Oil	170 Ltrs	D/G	0.00 m/t	
D/G LO	12 Ltrs	Boiler	0.00 m/t	
		Total	0.00 m/t	
Fresh Water				
Produced	25 m/t			
Consumed	5 m/t			
Low BN ROB				
M/E LO	12935 ltrs.	HSFO	1705.3 m/t	
M/E Sump TK	16600 ltrs.	LSDO	0.0 m/t	
Cyl. Oil	16380 5630 ltrs.	LSGO	379.8 m/t	
D/G LO	4627 ltrs.	TOTAL DO	379.8 m/t	
FW	135 m/t			

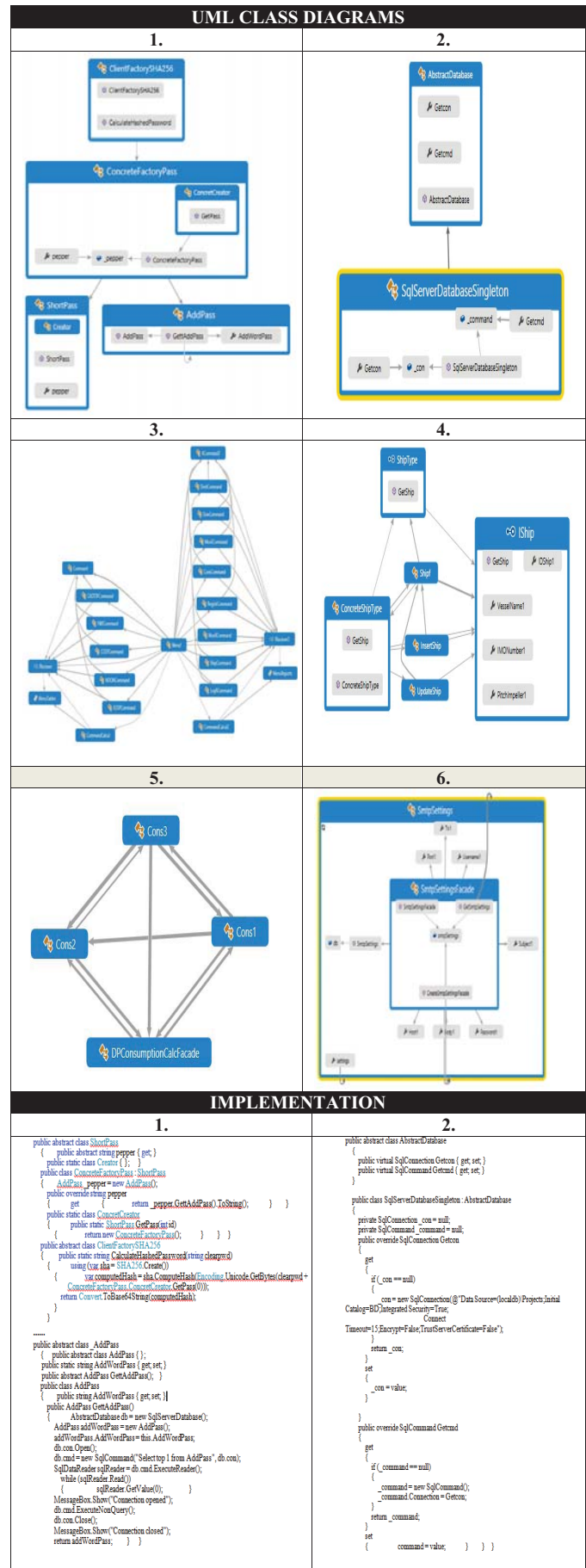
- Send Mail to send generated reports: a menu which can provide daily reports to the office.

B. CLASS DIAGRAMS AND IMPLEMENTATION.

Table 4 presents both a class diagram of each of the DPs used to implement software based on a combination of these patterns as well as their implementation.

- For Working with User Registry Encryption and Decryption Data, DP AbstractFactorySHA256 based on Abstract Factory Design Pattern is developed.
- A DPSingleton Pattern based on the Singleton Design Pattern is developed to access RDB.
- A DPCommandMenu Pattern based on Command Design Pattern is developed for menus.
- A DPAbstractFactoryShip Pattern based on the AbstractFactory Design Pattern is developed for entering and modifying data in RDB.
- For inserting and modifying data and computing, a DPConsumptionCalcFacade Pattern based on Facade Design Pattern was developed.
- To send mail, a DPFacadeMail Pattern based on Facade Design Pattern was developed.

TABLE 4 CLASS DIAGRAMS AND IMPLEMENTATION



3.	4.
<pre>//LIST1 enum MenuTables{...} interface IReceiver{...} abstract class Command{...} class NOONCommand : Command{...} class EOSPCCommand : Command{...} class COSPCCommand : Command{...} class CASTOFCommand : Command{...} class FWECommand : Command{...} class CommandCalcul : IReceiver{...} //LIST2 enum MenuReports{...} interface IReceiver2{...} abstract class ACommand2{...} class RegistrCommand : ACommand2{...} class ShipCommand : ACommand2{...} class DestCommand : ACommand2{...} class GravCommand : ACommand2{...} class MovBCommand : ACommand2{...} class MovFCommand : ACommand2{...} class LogBCommand : ACommand2{...} class ConsCommand : ACommand2{...} class CommandCalcul2 : IReceiver2{...}</pre>	<pre>public interface IShip{...} public interface ShipType{...} public class ConcreteShipType : ShipType{...} public IShip GetShip(tool b){...} public class InsertShip : IShip{...} { int IDShip1 (get; set;) string IPONumber1 (get; set;) string VesselName1 (get; set;) double PitchImpeller1 (get; set;) int IShip.IDShip1 (get; set;) string IShip.IPONumber1 (get; set;) string IShip.VesselName1 (get; set;) double IShip.PitchImpeller1 (get; set;) void IShip.GetShip(){...} } public class UpdateShip : IShip{...} { int IDShip1 (get; set;) string IPONumber1 (get; set;) string VesselName1 (get; set;) double PitchImpeller1 (get; set;) int IShip.IDShip1 (get; set;) string IShip.IPONumber1 (get; set;) string IShip.VesselName1 (get; set;) double IShip.PitchImpeller1 (get; set;) void IShip.GetShip(){...} } }</pre>
5.	6.
<pre>public class DPConsumptionCalcFacade { public double TimeHRS1 { get; set; } public string MERevCntrBegin { get; set; } public string MERevCntrEnd { get; set; } public string ObsDist { get; set; } public class ConsumptionC13 : DPConsumptionCalcFacade{...} public class ConsumptionC13Facade : ConsumptionC13{...} public class ConsumptionC111 : DPConsumptionCalcFacade{...} public class ConsumptionC111Facade : ConsumptionC111{...} public class ConsumptionC112 : DPConsumptionCalcFacade{...} public class ConsumptionC12Facade : ConsumptionC112{...} AbstractDatabase db = new SqlServerDatabaseSingleton(); public DPConsumptionCalcFacade InsertConsumption(){...} public DPConsumptionCalcFacade UpdateConsumption(){...} }</pre>	<pre>public class SetSettings { public string Host1 { get; set; } public int Port1 { get; set; } public string UserName1 { get; set; } public string Password1 { get; set; } public string From1 { get; set; } public string To1 { get; set; } public string Subject1 { get; set; } public string Body1 { get; set; } public SetSettings settings { get; set; } AbstractDatabase db = new SqlServerDatabaseSingleton(); public class SetSettingsFacade : SetSettings{...} { private readonly SetSettings setSettings; public SetSettingsFacade(){...} public void CreateSetSettingsFacade(){...} public SetSettings GetSetSettings(){...} } }</pre>

V. CONCLUSION AND FUTURE PROGRESS

Our approach to build a class of Software Systems through a Combination of DPs is presented in this paper. The patterns used are from the GOF group. DPs are a useful tool in solving repetitive problems that occur in real-world programming.

The paper also suggests a Compound Design Patterns (CDP) classification, summarizes the existing practice in the field of combined use of design patterns and the benefits of the combined use of DPs when building software.

Real results from the use of the approach developed are commented: an IS for the purposes of Maritime Transport is implemented using the approach.

Our research continues in the direction of new experiments and assessment of the developed approach. The design patterns used in the approach are usually evaluated by measuring the maintenance of the software, the evaluation being based on the following indicators: Cyclomatic Complexity, Class Coupling, Maintainability Index, Depth in Inheritance and Lines of Code [17, 18]. Other qualitative indicators that could be monitored are: manageability, performance and efficiency. The development and use of DPs does not have a negative impact on productivity. But they should be applied in software applications with caution. The software in which DPs are used has fewer chances of productivity problems in the supported code, even if they occur; they are easier to remove in code with greater complexity. Efficiency is measured by measuring the execution time.

REFERENCES

- [1] <https://vinodnambiar.wordpress.com/2011/09/29/service-composition>
- [2] <http://blog.e-zest.com/design-pattern-combination-strategy-with-factory-method>
- [3] <http://www2.fiit.stuba.sk/~filkorn/docs/EEICT2004.pdf>
- [4] <http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.378.779&rep=rep1&type=pdf>
- [5] <http://andrey.moveax.ru/design-patterns/oop/>
- [6] <https://www.codeproject.com/Articles/13229/Implementing-Observer-Strategy-and-Decorator-Design>
- [7] Design Patterns - education and classification_cha.pdf
- [8] Erich Gamma et al, *Design Patterns: Elements of Reusable Object-Oriented Software*, ISBN: 0201633612, Addison-Wesley Publ. Co., January 15, 1995
- [9] https://www.researchgate.net/publication/313578261_Evaluation_of_Software_Design_Pattern_on_Mobile_Application_Based_Service_Development_Related_to_the_Value_of_Maintainability_and_Modularity
- [10] <https://www.cs.rug.nl/paris/papers/EPLOP11.pdf>
- [11] http://serviceorientation.com/soaglossary/compound_design_pattern
- [12] https://www.duo.uio.no/bitstream/handle/10852/61661/theepank_main_document.pdf?sequence=32&isAllowed=y, 2018
- [13] <https://pdfs.semanticscholar.org/1833/f5927952d8beaad298306228c8e8c5cdc1c9c.pdf>
- [14] <http://www.righthandtech.com/controller-development.php>
- [15] <https://enterpriseprogrammer.com/2012/07/04/adapter-factory-design-pattern/>
- [16] <http://homepages.dcc.ufmg.br/~figueiredo/disciplinas/papers/jss2014cacho.pdf>
- [17] https://www.duo.uio.no/bitstream/handle/10852/60024/FinalThesis_Farooq.pdf?sequence=1&isAllowed=y, 2017
- [18] <https://dailydotnettips.com/understand-the-complexity-and-maintainability-of-your-coding-using-code-metrics-in-visual-studio/>